Decentralized Control System Reduction to Single Agent Reinforcement Learning

Dylan Kim

Date

Abstraction

One widely studied way to approach the decentralized partially observable system is based on Reinforcement Learning(RL). As the RL algorithm also gets generalized as a Multi-agent RL, training the model to converge to the solution becomes more unstable due to inherent limitations of the system, such as Non-stationarity. Hence, a way to reduce a multi-agent system into a single agent system using ordinary single agent RL has been proposed. This paper aims to apply two reduction algorithms to a decentralized system where multiple agents collaborate to find the solution and analyze the performance by comparing the two algorithms via implementation and simulation.

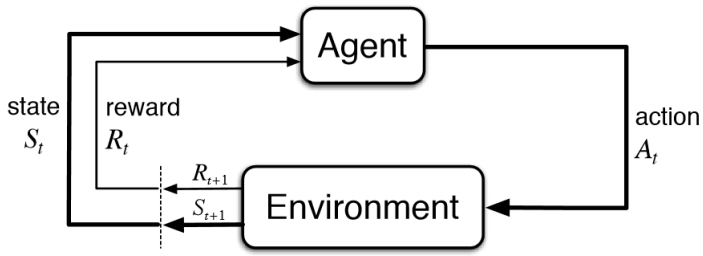
1 Background

This section will provide the theoretical background for the experiments that were conducted. The initial part will introduce an abstracted model of the system as a Markov Decision Process and explain its relationship with game theory. Then, the formal definition of the system's solution and its requirements will follow. Finally, Multi-agent Reinforcement Learning(RL) is presented as a solution; focuses on the algorithm that reduces MARL into single agent RL.

1.1 Agents-Environment Interaction

One way to solve the system where the decision makers aim to achieve a certain goal is to cast the problem into Markov Decision Processes(MDP). MDP proposes to abstract those systems into three alternate signal convey between the decision maker, called agent, and the world in which the agent can operate, called environment:

1. **State**: Current condition of the environment, which the agent uses to make a decision
2. **Reward:** A numerical signal provided by the environment to the agent
3. Agent-controllable **Action** that influences the environment.



[Figure 1. Key Concept of MDP]

Suppose the agent receives some state S_t at discrete time step t. Then, it selects an action A_t based on that state and sends it to the environment. In the next step, as a consequence, the environment returns some numerical reward R_{t+1} with a new state S_{t+1}. This numerical reward allows the agent to evaluate the quality of taking the action A_t at state S_t; the positive reward encourages the agent to take the same action while the negative reward encourages it to avoid. As the agent repeats this process, takes actions and records their quality, it aims to find the best actions for all states that yield the highest cumulative rewards.

The "cumulative rewards" is denoted as *return* G_t, a function of all future rewards.

$$\begin{aligned} G_t :=& R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \cdots \gamma^{T-1} R_T \\ =& \sum_{k=0}^{T-1} \gamma^k R_{t+k+1} \end{aligned} \tag{1}$$

where discount factor, $\gamma \in [0, 1]$, determines the agent's preference between immediate reward and long-term; the agent will prioritize long-term reward as γ approaches 1. T refers to the terminal time step. T can be infinite if the system does not have a clear termination, but then γ must not be 1 because $G_t = R_{t+1} + R_{t+2} + R_{t+3}...$ may become infinite.

The agent then quantifies the long-term quality of taking specific action in a given state based on the return it received. The long term quality is called *Value*; it can be classified depending on its input, either the state or the state-action pair. This paper will focus on *(State) Action Value* $q(s, a)$, the value with respect to the state-action pair.

Values for all possible state-action combination are defined based on a function called a *Value Function*; the agent will greedily select the actions that yield the highest values from the value function. This indicates that the value function forms an agent's *policy* π for which action to take in each state. The value of taking action a in a state s under policy π, denoted as $q_\pi(s, a)$, is defined as:

$$\begin{aligned} q_\pi(s, a) :=& \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \\ =& \mathbb{E}_\pi\Big[\sum_{k=0}^{T-1} \gamma^k R_{t+k+1} \Big| S_t = a, A_t = a\Big] \end{aligned} \tag{2}$$

where $\mathbb{E}$ is denoted as *expected* value.

If there is more than one way to estimate the value, one may give a higher expected return than the other for every state. Then, there is always at least one value function that is better than or equal to all other policies. The *Optimal Value Function* $q_*(s, a)$ represents the maximum expected return achievable by any policy, and the corresponding policy π_* is called the *Optimal Policy.*

$$q_*(s, a) := \max_\pi q_\pi(s, a) \tag{3}$$

The *Brute-Force* Search for the optimal value function by enumerating all possible value functions has exponential time complexity [1, pg 39]. Dynamic programming provides a computationally efficient way to find the optimal value function by breaking the problem into smaller subproblems. The following dynamic programming approach allows the agent to compute the value of the current state based on the value of the next state, enabling

recursive computation of the optimal policy:

$$\begin{aligned} G_t &:= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \cdots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \tag{4}$$

Then,

$$\begin{aligned} q_\pi(s,a) &:= \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, a) | S_t = s, A_t = a] \end{aligned} \tag{5}$$

This indicates that the value can be calculated with a single calculation, provided the value *for the next state*. Furthermore, based on this equation, Equation 3 can yield the following:

$$q_*(s,a) := \max_\pi q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} q_*(S_{t+1}, a') | S_t = s, A_t = a] \tag{6}$$

where $\max_{a' \in \mathcal{A}(S_{t+1})} q_*(S_{t+1}, a')$ means the optimal value function of the next state. i.e. the value of taking action that gives the highest value at state S_{t+1}.

This equation is called the Bellman Optimality Equation. It states that the value of the current state depends on the immediate reward and the maximum value of the next state, providing a recursive framework for computing the optimal policy [2]. Therefore, when the agent reaches the terminal state, it redefines the value function of the smallest subproblem, which is the immediately preceding state. Then, it redefines the value function for the next smallest subproblem with the solution of the smallest subproblem. It eventually redefines all states, from the terminal state to the initial state recursively.

1.2 Partially Observable Stochastic Game(POSG)

This section will expand the understanding of single agent-environment interaction into multi-agent system partial observability. One way to mathematically define this system is to express such interaction as a game-theoretical model, called *game.* Various game models form a hierarchy in terms of the complexity of states and agents system, this paper will focus on the conceivably generalized form, which is a Partially Observable Stochastic Game, abbreviated as POSG.

The biggest characteristics of POSG against more specified models are multi-agentness and partial observability. In POSG, multiple agents interact with the environment to achieve a shared or individual objective. At each time step, actions from all agents will simultaneously affect the environment. In a partially observable system, agents do not have access to the full state of the environment; instead they receive incomplete information about the state and actions, denoted as *observation*, from the observation function $\mathcal{O}_i$. The observation function can model the uncertainty in the agent's perception by returning the probability distribution over possible observations given the current state and action.

⟨**Partially Observable Stochastic Game** ⟩

$Def.$ Partially Observable Stochastic Game consists of...

1. Finite set of agents $I = \{1, 2 \dots n\}$
2. Finite set of states S, with subset of terminal states $\bar{S} \subset S$

For each agent $i \in I$

3. Finite set of actions A_i
4. Reward function $\mathcal{R}_i : S \times A \times S \to \mathbb{R}$, where $A = A_1 \times \cdots \times A_n$
5. State transition probability function $\mathcal{T} : S \times A \times S \to [0, 1]$ such that

$$\forall\ s \in S, a \in A : \sum_{s' \in S} \mathcal{T}(s, a, s') = 1 \tag{7}$$

6. Initial state distribution $\mu : S \rightarrow [0, 1]$ such that

$$\sum_{s \in S} \mu(s) = 1 \quad \text{and} \quad \forall\ s \in \bar{S} : \mu(s) = 0 \tag{8}$$

7. Finite set of observation O_i
8. Observation function $\mathcal{O}_i : A \times S \times O_i \rightarrow [0, 1]$ such that

$$\forall\ a \in A, s \in S : \sum_{o_i \in O_i} \mathcal{O}_i(a, s, o_i) = 1 \tag{9}$$

The game starts in the initial state $s_0 \in S$ sampled by μ. At each time step t, the game is in state s^t and each agent chooses a decision $a_i^t \in A_i$. This action is taken based on the action probability distribution defined in its policy $\pi_i(a_i^t|h_i^t)$. $h_i^t = (o_i^1 ... o_i^t)$ refers to an observation history, all of the agent's past observations until the most recent observation. Given a joint action $a^t = (a_1^t ... a_n^t) \in A_1 \times ... \times A_n$, the game transitions to the next state $s^{t+1} \in S$ with the probability defined from the transition function $\mathcal{T}(s^{t+1}|s^t, a^t)$. Then, all agents receive their rewards $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$ and observations $o_i^{t+1} \in O_i$ from observation function $\mathcal{O}_i(a^t, s^{t+1}, o_i^{t+1})$. This cycle is repeated until the system reaches terminal state $s^t \in \bar{S}$ or counter of the cycle exceeds the predefined scalar.

POSG is often classified with respect to the reward system. One example is a zero-sum game, the rewards for all agents sum to zero, meaning that if one agent gets rewarded, the other agents get punished. This model suits a system where two agents compete, like rock, scissors, paper. Since this paper focuses on the collaboration of agents rather than competition, it highlights a common reward game (a.k.a common payoff game), where all agents receive the same reward function. This particular model is also known as Decentralized Partially-Observable Markov Decision Process(Dec-POMDP).

1.3 Observation Function

From a game theoretical point of view, partial observability and observation functions are not static but are defined dynamically depending on the system. Several examples are introduced:

1. **Stochastic Game:** Stochastic game is POSG model without partial observability. Thus, its observation function can be defined as *Full Observation:* $o_i^t = (s^t, a^{t-1})$
2. **Limited View Region:** Agents may have only limited view towards the system, thus observe only part of the state and part of the joint action: $o_i^t = (\bar{s}^t, \bar{a}^t)$ where $\bar{s}^t \subset s^t$ & $\bar{a}^t \subset a^t$

Observation function can apply uncertainty to the observation by assigning non-zero probability to multiple possible observations. Furthermore, it can simulate the communication between agents. A communication message can be viewed as a particular type of action that can be received by other agents without modifying the environment. In this case, the action space of the agent i is redefined as $A_i = X_i \times M_i$ where X_i is a set of actions that modify the environment and M_i is a set of communication actions, and so the joint action is updated as $\big((a_1^t, m_1), \ldots (a_n^t, m_n)\big)$.

1.4 Optimality Criterion

Since all agents in Dec-POMDP system receive the common reward, a straightforward definition of the solution is to maximize the expected return. This section will explain the game-theoretical logic behind this. First, consider the following terminologies.

⟨ **Best Response** ⟩

Let the set of policies for all agents except agent i be $\pi_{-i} = (\pi_1 ... \pi_{i-1}, \pi_{i+1} ... \pi_n)$. Then, a *best response* for agent i to π_{-i} is a policy π_i that maximizes the expected return for i when played against π_{-i}, denoted as $BR_i(\pi_{-i})$. Note that best response is not unique, $BR_i(\pi_{-i})$

may contain more than one policy. Since best response defines the optimal strategy for an agent in response to the strategies of others, all solutions in this chapter can be expressed based on this.

⟨ **Nash Equilibrium** ⟩

def A Nash Equilibrium is a set of policies $\pi = (\pi_1, ..., \pi_n)$ such that for each agent i, π_i is a best response to π_i

$$\forall i,\ \pi \in BR_i(\pi_{-i})$$

A Nash equilibrium of a game is a set of strategies (or policies) for each agent in the game such that no agent can achieve a higher expected return by unilaterally changing its strategy, assuming other agents' policies stay fixed [3, pg 68]. This means that each agent's local policy in the Nash Equilibrium is a best response against the others'.

One important point is that a game may have multiple Nash Equilibria where each equilibrium yields different expected returns for the agents. Since there is no such *optimal* solution but *optimum* solutions, it is important to consider which equilibrium the agent should aim to converge to.

⟨ **Pareto Dominance and Optimality** ⟩

Def. A joint Policy π is Pareto dominated by another joint policy π' if

$$\forall\, i :\ U_i(\pi') \geq U_i(\pi) \ \text{ and } \ \exists\, i :\ U_i(\pi') > U_i(\pi) \tag{10}$$

where $U_i(\pi)$ represents the expected return for agent i. A joint policy π is *Pareto Optimal* if it is not Pareto-dominated by any other joint policy. Then the expected returns of π are referred to as *Pareto-Optimal.*

In other words, the joint policy is Pareto-optimal if there is no other joint policy that improves the expected return for at least one agent without reducing the expected return for any other agents. Pareto Optimality allows multiple Nash Equilibria to be compared and

determined which one is better.

In Dec-POMDP, where all agents share a common reward function, Pareto Optimality ensures that the joint policy maximizes the expected return for all agents without favoring any individual agent. In other words, all Pareto optimal policies for a common-reward game achieve the same expected return [3, pg 77]. Therefore, as long as the agent converges to Nash equilibria, it is Pareto-optimal. This follows that the solution to the Dec-POMDP is to maximize the expected return, without considering other possible equilibria.

1.5 Reinforcement Learning

Reinforcement Learning(RL) is a learning algorithm that derives the optimal policy for the game using the concept from section 1.1. At each time step, the agent selects an action based on a selection algorithm. This paper will use ε-selection, where the agent chooses actions possessing the largest values with the probability of $1-\varepsilon$, or taking a random action, expecting to find the better option with probability of $\varepsilon \in [0, 1]$. As the agent receives the reward from the environment, it updates the values corresponding to the action. The agent repeats this process until the system reaches some terminal states. A complete run of this loop from the initial to the terminal state is called an *episode*. Then, the generated data produced from multiple episodes will fine-tune the agent's policy, aiming to converge to (Pareto-Optimal) Nash Equilibrium.

$\langle$ **Q Learning** $\rangle$

Q-Learning(QL) is a model-free RL algorithm that learns the optimal policy by iteratively updating the Q-table, which stores the expected cumulative rewards for each state-action pair. In QL, the agent constructs the policy directly by filling in a Q table based on the interaction with the environment. The Q table is a table of which rows and columns represent the states and actions, respectively. And the Q values $Q(s, a)$, which represent

the state-action values, are stored in each entry. Updating system of Q value is based on the Dynamic Programming paradigm, once the agent reaches the terminal state, it back-tracks its steps to its initial state. During this, it updates the Q values of all (state, action) combinations it visited using the Bellman Optimality Equation:

$$Q^{new}(s,a) = Q^{old}(s,a) + \alpha\Big[R_{t+1} + \gamma \cdot \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1},a) - Q^{old}(s,a)\Big] \tag{11}$$

where α and γ refer to learning rate and discount factor ($\alpha, \gamma \in [0,1]$) [4]. A learning rate determines how fast the algorithm learns, and the discount factor determines whether the model focuses more on the short term future reward over the long term, or vice versa.

Q Learning is guaranteed to converge to the optimal solution in two conditions [3, pg 33, 35]:

1. All state-action combinations $(s,a) \in S \times A$ must be tried an infinite number of times during learning.
2. α must be reduced over time in a way that satisfies the following conditions.

$$\forall s \in S,\ a \in A: \quad \sum_{k=0}^{\infty} \alpha_k(s,a) \to \infty \ \text{ and } \sum_{k=0}^{\infty} \alpha_k(s,a)^2 < \infty \tag{12}$$

where $\alpha_k(s,a)$ denotes the learning rate when applying Equation 12 k^{th} selection of action a in state s. The left-hand sum ensures that the learning rate is large enough to overcome initial learning conditions, while the right-hand sum ensures that the sequence will converge at a certain rate.

1.6 Multi-agent Reinforcement Learning (MARL)

Multi-agent RL(MARL) extends single-agent RL to environments where multiple agents interact, introducing challenges such as non-stationarity, partial observability, and

interdependent rewards. In MARL, n agents individually select actions based on their local policy. Then, *joint action* $a = (a_1^t, ..., a_n^t)$, a collection of all actions, influences the environment. Consequently, agents receive individual observations and rewards (recall that agents receive the same reward in the Dec-POMDP system). Then, similar to the single agent RL, this process is repeated until the system reaches the terminal state; the set of episodes will improve a joint policy $\pi = (\pi_1, ...\pi_n)$, a collection of all local policies. MARL algorithm learns the optimal joint policy for the set of agents in POSG by maximizing the agents' cumulative rewards.

1.7 Single Agent RL Reductions

The most basic approach to solving a multi-agent system using RL is to reduce it to a single-agent learning problem. This section will introduce two such approaches: *central learning* where a single agent takes action from the joint action space and learns the central policy that chooses the action for each agent; and *independent learning* where all agents apply a single RL algorithm solely to learn their local policies, essentially ignoring the presence of other agents.

⟨ **Central Q Learning** ⟩

In central learning, a.k.a joint action learning, a central controller receives the local observations from all agents, then tells them which action to take by selecting a joint action from space $A = A_1 \times ...A_n$. As the agent aims to maximize the value for joint action, the central policy can converge to the solution, since it will provide the joint policy that yields the largest value for all states. However, central learning has some critical limitations in practice.

The first limitation is to transform the joint reward $r = (r_1, ..., r_n)$ into a single scalar reward. Recall that each agent in a multi-agent system receives individual rewards. Since the

central learning trains a single central learning, this requires a specific reducing algorithm of the joint reward to a single scalar. For most of POSG, such as zero sum and general sum game, the way to scalarize the joint reward is unclear. However, applying central learning to the common reward system, is extremely simple since the reward is shared among all agents; global reward r is equal to local reward r_i.

The second limitation is that the size of the action space of the joint action learner will increase exponentially in the number of agents. For example, the action space of joint action learner of the system containing 3 agents with 6 actions will be $6^3 = 216$. Therefore, central learning would effectively work only for smaller systems.

The third limitation came from the physical limitation of the Multi-agent system. Agents are often localized entities that are physically or virtually distributed; a joint action learner cannot communicate the solution to each agent. Therefore, such a system requires local policies that act based on agents' local observation, which the next subsection will introduce.

One example of central learning is *central Q-learning* (CQL). Recall that original Q Learning guarantees the convergence. Then, as central learning reduces multi-agent system into single agent, using CQL guarantees to learn optimal policy for common-reward stochastic game(POSG without partial observability) [3, 95]. In this case, the common reward principle forces the policy to be Pareto-optimal because there is no other policy that achieves higher return for "any" agent. Then, individual policies decomposed by the central optimal policy are indeed Nash Equilibria.

CQL for stochastic game

1. Initialize: $Q(s, a) = 0$ for all $s \in S$ and $a \in A = A_1 \times \ldots \times A_n$.
2. Repeat for every episode:
3. For $t = 0, 1, 2, \ldots$:
   (a) Observe current state s^t
   (b) With probability ϵ: choose random joint action $a^t \in A$
   Otherwise choose joint action $a^t \in \arg\max_a Q(s^t, a)$
   (c) Apply joint action a^t, observe rewards $r_1^t, \ldots, r_n^t$ and next state s^{t+1}
   (d) Transform $r_1^t, \ldots, r_n^t$ into scalar reward r^t
   (e) Update Q-value:

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[r^t + \gamma \max_a Q(s^{t+1}, a) - Q(s^t, a^t) \right]$$

⟨ **Independent Learning** ⟩

In independent learning, each agent i learns its policy π_i using only its local history of observations, actions, and rewards, while *ignoring the existence of other agents.* This means that agents do not observe or use information about other agents but includes other agents' actions to the dynamics of the environment.

The strength of independent learning is that the limitations introduced for central learning are not applied to independent learning. However, independent learning can be significantly affected by nonstationarity caused by concurrent learning of all agents. For example, in the agent i's perspective, the possible transition from state s may be nonstationary as agent j changes its action from the previous episode. This can cause unstable learning, and the algorithm may not converge to any game solution.

This paper will use independent learning based on Q-learning, called *independent Q-learning*:

IQL for stochastic game

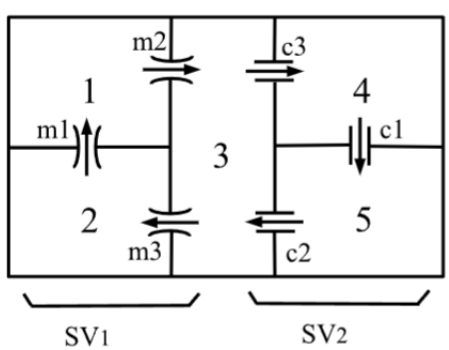
1. Initialize: $Q_i(s, a_i) = 0$ for all $s \in S$, $a_i \in A_i$.
2. Repeat for every episode:
3. For $t = 0, 1, 2, \ldots$:
   (a) Observe current state s^t.
   (b) With probability ϵ: choose random action $a_i^t \in A_i$.
   Otherwise: choose action $a_i^t \in \arg\max_{a_i} Q_i(s^t, a_i)$.
   (c) (meanwhile, other agents $j \neq i$ choose their actions a_j^t).
   (d) Observe own reward r_i^t and next state s^{t+1}.
   (e) Update Q-value:

$$Q_i(s^t, a_i^t) \leftarrow Q_i(s^t, a_i^t) + \alpha \left[r_i^t + \gamma \max_{a_i} Q_i(s^{t+1}, a_i) - Q_i(s^t, a_i^t) \right]$$

Note that this algorithm controls agent i and the same algorithm exists for all agents.

1.8 Cat and Mouse Maze Problem

This section will propose a decentralized system introduced by Toshimitsu Ushio and Tatsushi Yamasaki, called Masze for cat and mouse problem [5].



[Figure 2. Maze for cat and mouse with 2 supervisors]

There are 5 rooms partitioned by doors. Each door is one-way, used by a cat or a mouse exclusively. Mouse and cat locate 2 and 4, denoted as $(2, 4)$, at the initial state.

If the door in front of the cat or mouse is opened, then they will move at that time step. The objective is to make cat and mouse travel the system while avoiding (3,3) where they encounter at room 3. The system will be *terminated* when (3,3) happens, or *truncated* when 30 time steps are passed without occurrence of (3,3).

To reach the solution, two supervisors are allowed to control each kind of event, c_i's and m_i's (in this paper, the term "supervisor" is used interchangeably with the "agent" from RL). SV_1 (mouse's supervisor) can observe occurrence of $\{m_1, m_2, m_3, c_2, c_3\}$, existence of cat or mouse at rooms 1,2,3, and can control $\{m_1, m_2, m_3\}$. On the other hand, SV_2 (cat's supervisor) can observe occurrence of $\{m_2, m_3, c_1, c_2, c_3\}$, existence of cat or mouse at rooms 3,4,5, and can control. $\{c_1, c_2, c_3\}$. Since c_2 is uncontrollable for SV_1, it cannot prevent the cat entering room 3 from room 2. As a consequence of the joint action, supervisors will receive local observation of the system. Suppose the current state of the system is (2,4) and event m_1 is selected. Since SV_1 does observe m_1, it would receive its local state of (1,4). However, since SV_2 cannot observe m_1 is happening, its local state would maintain (2,4).

2 Method

This section will present the common settings along the experiments, including how the environment was implemented and the reward system. The motivation, variables, and result of each experiment will be introduced in the next section.

The environment is implemented based on the Python *gymnasium* library. Although specific scalars for the numerical reward differ between experiments, the overall structure is consistent. Note that since Ushio and Yamasaki did not specify which feature moves first, each experiment is conducted based on two environments: 1. cat always moves first (specific case); 2. each feature has 50% to move first(general case). Since the second situation is more general, the results are all based on this system.

With respect to the RL reward assignment convention, the reward system is designed to incentivize *what* the system wants to achieve, not *how* the system wants to achieve. The default reward system is defined as follows:

1. When the cat and mouse enter the room 3 at the same time, both receive -100
2. If both the cat and mouse have visited the room 3 without collision, both receive 10. This can happen multiple times

To implement the second criteria, both cat and mouse have boolean flags tracking whether they have visited room 3. If both flags are true, then they get positive reward and the flags reset to false.

This paper conducted two experiments with CQL and another two with IQL. Then, the performance of each model is evaluated by applying the model to the system simulation as well as its average return values for each unit epoch calculated. These return values are plotted to further examine which algorithms converge the fastest to the solution.

3 Results

3.1 Exp 1: CQL with Partial Observation

Recall the CQL algorithm introduced in Section 1.7. Observe that the joint action agent chooses the action based on the state itself, not based on partial observation received by other agents. To guarantee application of partial observability, this experiment proposes another algorithm that the joint action agent chooses action based on the Cartesian product of agents' partial observation spaces as follows:

CQL for POSG

1. Initialize: $Q(s,a) = 0$ for all $s \in \hat{S} = O_1 \times ... \times O_n$ and $a \in A = A_1 \times \ldots \times A_n$.
2. Repeat for every episode:
3. For $t = 0, 1, 2, \ldots$:
   (a) Observe the current state $s^t = (o_1^t, ...o_n^t)$ where $o_i^t \in O_i$
   (b) With probability ϵ: choose random joint action $a^t \in A$
   Otherwise choose joint action $a^t \in \arg\max_a Q(s^t, a)$
   (c) Apply joint action a^t, observe rewards $r_1^t, \ldots, r_n^t$ and next state $s^{t+1} = (o_1^{t+1}, ..., o_n^{t+1})$
   (d) Transform $r_1^t, \ldots, r_n^t$ into scalar reward r^t
   (e) Update Q-value:

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[r^t + \gamma \max_a Q(s^{t+1}, a) - Q(s^t, a^t) \right]$$

Since each mouse and cat can be in 3 rooms, the total number of states is $3 \times 3 = 9$. However, in the perspective of SV_1, it cannot distinguish the cat from being in room 4 and room 5, as it cannot observe c_1. Thus, it cannot distinguish (1,4) & (1,5), (2,4) & (2,5), and (3,4) & (3,5). Similarly, SV_2 cannot distinguish (1,3) & (2,3), (1,4) & (2,4), and (1,5) & (2,5). Therefore, each supervisor has six different local observations. Furthermore, in each state, the cat and mouse have 2 possible actions, closing and opening the door. Therefore, Q-table is implemented as $6^2 \times 2^2 \Rightarrow 36 \times 4$ table.

To find the minimum number of epochs for guaranteed convergence, this algorithm increased the number of epochs from 100 to 5000, incremented by 100. Then, at each condition, the average expected return of 30 time steps for 10 trainings was plotted. Each training set $\gamma = 0.9$ and $\alpha = 0.01$. Recall that this is based on 50-50 entry model.

The hyperparameters were set as follows:

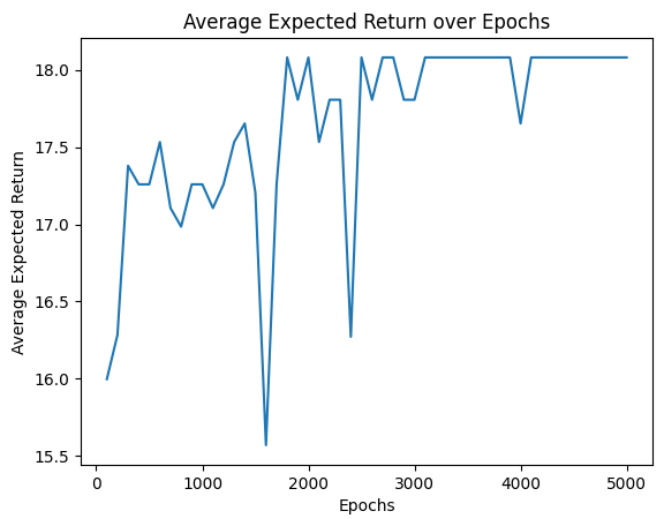
- $\alpha = 0.01$
- $\gamma = 0.9$

- $\epsilon = 0.1$

⟨ **Result** ⟩

Epochs	Return	Epochs	Return	Epochs	Return	Epochs	Return	Epochs	Return
100	15.998	1100	17.105	2100	17.532	3100	18.079	4100	18.079
200	16.283	1200	17.258	2200	17.806	3200	18.079	4200	18.079
300	17.378	1300	17.532	2300	17.806	3300	18.079	4300	18.079
400	17.258	1400	17.652	2400	16.271	3400	18.079	4400	18.079
500	17.258	1500	17.204	2500	18.079	3500	18.079	4500	18.079
600	17.532	1600	15.570	2600	17.806	3600	18.079	4600	18.079
700	17.105	1700	17.258	2700	18.079	3700	18.079	4700	18.079
800	16.984	1800	18.079	2800	18.079	3800	18.079	4800	18.079
900	17.258	1900	17.806	2900	17.806	3900	18.079	4900	18.079
1000	17.258	2000	18.079	3000	17.806	4000	17.652	5000	18.079

[Figure 3. Table for Average Expected Returns for Exp 1]



[Figure 4. Average Expected Return vs Epochs for Exp 1]

According to Figure 4, the average expected return increases as the number of epochs increases. Furthermore, although the performance fluctuates at first, it becomes stable; since around 3500 epochs, most of the training successfully converged to the solution. To evaluate the performance with the large amount of epochs, 100 more models are trained with 10,000

epochs, and it converged to the optimum solution that yielded a return value of 18.08 with a probability of 94%.

This experiment questioned whether performance would be affected if the system had full observability, which motivated the next experiment.

3.2 Exp 2: CQL with Full Observation

This experiment reduced the number of rows of the Q-table to 9, which is the total number of states based on the full observability. Therefore, this experiment is implemented with the original CQL algorithm introduced in Section 1.7.

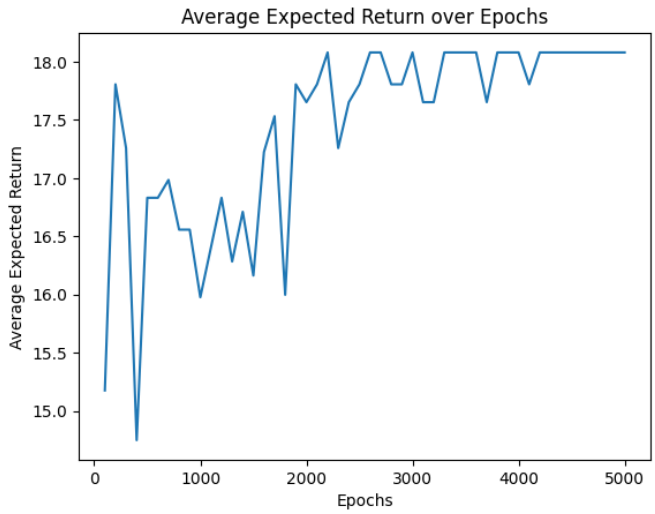
The hyper-parameters were set as follows:

- $\alpha = 0.01$
- $\gamma = 0.9$
- $\epsilon = 0.1$

⟨ **Result** ⟩

Epochs	Return	Epochs	Return	Epochs	Return	Epochs	Return	Epochs	Return
100	15.176	1100	16.404	2100	17.806	3100	17.652	4100	17.806
200	17.806	1200	16.831	2200	18.079	3200	17.652	4200	18.079
300	17.258	1300	16.283	2300	17.258	3300	18.079	4300	18.079
400	14.749	1400	16.711	2400	17.652	3400	18.079	4400	18.079
500	16.831	1500	16.163	2500	17.806	3500	18.079	4500	18.079
600	16.831	1600	17.225	2600	18.079	3600	18.079	4600	18.079
700	16.984	1700	17.532	2700	18.079	3700	17.652	4700	18.079
800	16.557	1800	15.998	2800	17.806	3800	18.079	4800	18.079
900	16.557	1900	17.806	2900	17.806	3900	18.079	4900	18.079
1000	15.977	2000	17.652	3000	18.079	4000	18.079	5000	18.079

[Figure 5. Table for Average Expected Returns for Exp 1]



[Figure 6. Average Expected Return vs Epochs for Exp 2]

The overall trend of the average expected return did not change much from the first experiment. Both started converging to the optimum solution around 2000 epochs and get stabilized as the number of epochs increases. Performance with the larger scale of epochs was similar as well; out of 100 models trained with 10,000 epochs, 92% of them converged to the solution with return value of 18.08. The reason behind these similarities will be introduced in the next section.

3.3 Exp 3: IQL with Default Reward System

This experiment simply replaces CQL with IQL. Similar to the experiment 1, each agent observes not the full state, but its projection to apply partial observability. Since each agent has 6 observations, it has the 6×2 Q-table, 2 representing the size of the action space.

IQL for POSG

1. Initialize: $Q_i(o_i, a_i) = 0$ for all $o_i \in O_i$, $a_i \in A_i$.
2. Repeat for every episode:
3. For $t = 0, 1, 2, \ldots$:
   (a) Observe current state s^t.
   (b) With probability ϵ: choose random action $a_i^t \in A_i$.
   Otherwise: choose action $a_i^t \in \arg\max_{a_i} Q_i(o_i^t, a_i)$.
   (c) (meanwhile, other agents $j \neq i$ choose their actions a_j^t).
   (d) Observe own reward r_i^t and next obesrvation o_i^{t+1}.
   (e) Update Q-value:

$$Q_i(o_i^t, a_i^t) \leftarrow Q_i(o_i^t, a_i^t) + \alpha \left[r_i^t + \gamma \max_{a_i} Q_i(o_i^{t+1}, a_i) - Q_i(o_i^t, a_i^t) \right]$$

This algorithm was trained with epochs from 10,000 to 100,000; every 10,000 epochs, the algorithm was trained 5 times, and the averages of their expected return were recorded. Since the IQL required more epochs(next section will introduce why), the number of test cases had to be decreased.

The hyper-parameters were set as follows:

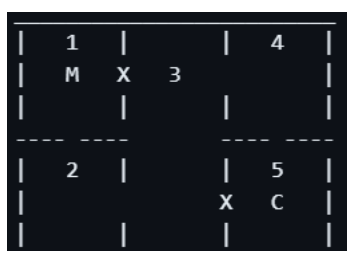
- $\alpha = 0.01$
- $\gamma = 0.9$
- $\epsilon = 0.1$

$\langle$ **Result** $\rangle$

Epochs	Return
10000	0
20000	0
30000	0
40000	0
50000	0
60000	0
70000	0
80000	0
90000	0
100000	0

[Figure 7. Table for Average Expected Returns for Exp 3]

The cat and mouse would receive negative reward if they encounter each other in room 3. Then, cumulative return being zero means they have never received a penalty nor a reward, just get stuck. According to the simulator, cat and mouse kept closing the door, refusing to enter the room 3 as follows:



[Figure 8. Image from the Exp 3 simulation]

This result brought up one hypothesis about the symmetry of the system. Observe that cat and mouse's behaviours in the system are symmetric, that switching them will not affect the context of the system. Recall that the condition that they can get a positive reward is also symmetric. These symmetries may cause them to learn completely the same behaviour, which was closing the door towards room 3 until the opponents enter first. This hypothesis motivated the last experiment with the updated reward function.

3.4 Exp 4: IQL with Updated Reward System

This section proposes a new reward system:

1. When the cat and mouse enter the room 3 at the same time, both receive -100
2. If the cat visited room 3 and the mouse moved at least once, both get 10 points.
3. If the mouse visited room 3 and the cat moved at least once, both get 10 points.
4. 2. and 3. can happen multiple times, and if both happen, then both get 20 points.

Unlike the first reward system, this reward system encourages the cat and mouse not only to visit room 3, but also to move even after that. Furthermore, as mentioned in the previous section, the criteria for positive reward demand asymmetric behavior towards the cat and mouse. Initially, 100 models were trained with 100,000 epochs.

The hyper-parameters were set as follows:

- $\alpha = 0.9$
- $\gamma = 0.9$
- $\epsilon = 0.1$

⟨ **Result** ⟩

Return	Counts
0.0	64
7.29	20
38.55	13
-81.0	1
-45.854	1
13.851	1

[Figure 9. Table for possible Returns and their counts]

Even though 64% of the sample showed the same tendency as previous algorithm, the model started making a new trend.

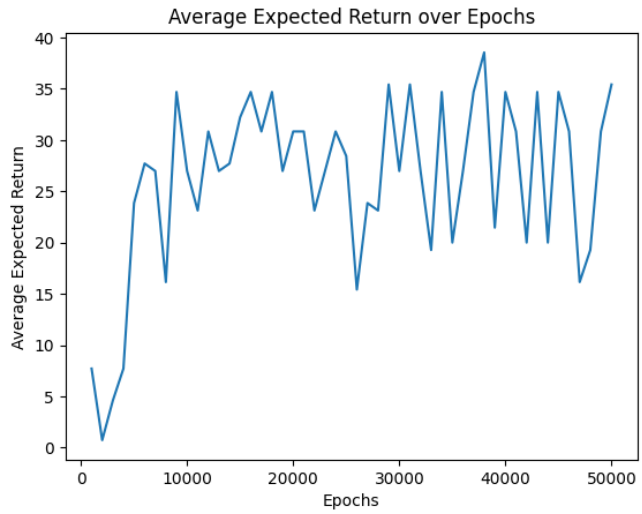
⟨ **Performance with lower learning rate** ⟩

Recall that the learning rate α is a hyperparameter that determines how fast the algorithm learns a new thing from Section 1.5. This also means how much the old value affects updating. If the learning rate is 1, then the old Q value does not affect, but is simply replaced by a new Q value. As the learning rate value decreases, the portion for the old value would increase, and updating would gradually reflect a new Q value. Fine-tuning the learning rate is related to the trade-off between the speed of learning and the stability of convergence. A high learning rate can lead to faster convergence but may also cause the overshoot or oscillate around the optimum solution. On the other hand, a low learning rate can result in slow convergence and may get stuck in suboptimal solutions. To expect a higher rate of convergence, the learning rate is tuned to 0.01.

Furthermore, to increase the sample size, this experiment made a trade-off by decreasing the epochs from 100,000 to 50,000 and increasing the amount of test for average return calculation from 5 to 10. Furthermore, the average returns are recorded every 1000 epochs.

Epochs	Return	Epochs	Return	Epochs	Return	Epochs	Return	Epochs	Return
1000	7.709	11000	23.129	21000	30.839	31000	35.423	41000	30.839
2000	0.729	12000	30.839	22000	23.129	32000	26.984	42000	20.003
3000	4.583	13000	26.984	23000	26.984	33000	19.274	43000	34.694
4000	7.709	14000	27.713	24000	30.839	34000	34.694	44000	20.003
5000	23.858	15000	32.225	25000	28.442	35000	20.003	45000	34.694
6000	27.713	16000	34.694	26000	15.419	36000	26.984	46000	30.839
7000	26.984	17000	30.839	27000	23.858	37000	34.694	47000	16.148
8000	16.148	18000	34.694	28000	23.129	38000	38.549	48000	19.274
9000	34.694	19000	26.984	29000	35.423	39000	21.461	49000	30.839
10000	26.984	20000	30.839	30000	26.984	40000	34.694	50000	35.423

[Figure 10. Table for Average Expected Returns for Exp 4]



[Figure 11. Average Expected Return vs Epochs for Exp 4]

According to Figure 9, updating the reward system dramatically improved performance. However, it did not guarantee the convergence, although it occasionally does. From 100 times of training sample with 100,000 epochs, it converged to the optimum solution with the probability of 72% with return value of 38.55.

4 Discussion

The experimental results presented in the previous section highlight both the potential and the challenges of reducing decentralized control systems with partial observability to single-agent RL frameworks. This section will discuss the following uncertainties raised during the experiments:

1. The reason why the performance of CQL with full observation was similar to that with partial observation.
2. Why did CQL show more stable convergence than IQL?

Following is the actual Q-table for joint action learner from Experiment 1, with each Q-value rounded up to the nearest hundredth.

Index	(0,0)	(0,1)	(1,0)	(1,1)	Index	(0,0)	(0,1)	(1,0)	(1,1)
0	21.56	21.82	-100.00	19.95	20	27.36	30.18	21.20	29.78
1	0.00	0.00	0.00	0.00	21	0.00	0.00	0.00	0.00
2	0.00	0.00	0.00	0.00	22	0.00	0.00	0.00	0.00
3	0.00	0.00	0.00	0.00	23	0.00	0.00	0.00	0.00
4	0.00	0.00	0.00	0.00	24	0.00	0.00	0.00	0.00
5	0.00	0.00	0.00	0.00	25	0.00	0.00	0.00	0.00
6	0.00	0.00	0.00	0.00	26	0.00	0.00	0.00	0.00
7	23.82	19.37	24.21	20.08	27	0.00	0.00	0.00	0.00
8	14.90	20.95	15.37	-100.00	28	0.00	0.00	0.00	0.00
9	0.00	0.00	0.00	0.00	29	0.00	0.00	0.00	0.00
10	0.00	0.00	0.00	0.00	30	0.00	0.00	0.00	0.00
11	0.00	0.00	0.00	0.00	31	0.00	0.00	0.00	0.00
12	21.43	21.09	19.70	22.46	32	0.00	0.00	0.00	0.00
13	0.00	0.00	0.00	0.00	33	0.00	0.00	0.00	0.00
14	0.00	0.00	0.00	0.00	34	25.93	19.94	20.77	26.88
15	0.00	0.00	0.00	0.00	35	19.08	-100.00	26.62	11.32
16	0.00	0.00	0.00	0.00					
17	0.00	0.00	0.00	0.00					
18	0.00	0.00	0.00	0.00					
19	20.98	21.20	21.86	21.20					

[Figure 11. Q-table from Experiment 1]

Although the size of the joint observation space is 36, it only filled eight feasible pairs. For example, it is impossible to have an observation pair as $((3,3),(3,4))$, thus all its values are zero. Consider a feasible row numbers: 0, 7, 8, 12, 19, 20, 27, 34, 35. Row number 27 will be added to this list since it represents (3,3); its entries are zeros even though it is a feasible state because the system terminates when (3,3) occurs. Each number represents one pair of partial observations in a following conversion algorithm. Consider row number n,

1. Find i, j based on $n = 6 \times i + j$ where $i, j \in [0, 5]$
2. i and j represent row number from Mouse's and Cat's observation functions, respectively

Index	Observations
0	(1,3)
1	{(1,4), (1,5)}
2	(2,3)
3	{(2,4), (2,5)}
4	(3,3)
5	{(3,4), (3,5)}

[Figure 12. Observation function for SV_1]

Index	Observations
0	{(1,3),(2,3)}
1	{(1,4), (2,4)}
2	{(1,5), (2,5)}
3	(3,3)
4	(3,4)
5	(3,5)

[Figure 13. Observation function for SV_2]

Then, each non-zero row can be converted as follows:

Row #	(i, j)	O_{mouse}	O_{cat}	$O_{\text{mouse}} \cap O_{\text{cat}}$
0	(0,0)	(1,3)	{(1,3), (2,3)}	(1,3)
7	(1,1)	{(1,4), (1,5)}	{(1,4),(2,4)}	(1,4)
8	(1,2)	{(1,4), (1,5)}	{(1,5), (2,5)}	(1,5)
12	(2,0)	(2,3)	{(1,3), (2,3)}	(2,3)
19	(3,1)	{(2,4), (2,5)}	{(1,4), (2,4)}	(2,4)
20	(3,2)	{(2,4), (2,5)}	{(1,5), (2,5)}	(2,5)
27	(4,3)	(3,3)	(3,3)	(3,3)
34	(5,4)	{(3,4), (3,5)}	(3,4)	(3,4)
45	(5,5)	{(3,4), (3,5)}	(3,5)	(3,5)

[Figure 13. Table for intersection of partial observations]

As the column "$O_{\text{mouse}} \cap O_{\text{cat}}$" represents, even though partial observation is applied to the joint action learner, it receives sufficient information from two agents to determine the actual state. Such conditions are called the jointly observable system, a.k.a. Decentralized Markov Decision Process(Dec-MDP)[1, pg 22]. Furthermore, since the Central Learning paradigm reduced multi-agent to a single-agent system, applying CQL to this problem can be considered as a reduction from Dec-POMDP to MDP. Recall that Q-Learning is guaranteed to converge to the optimal solutions. Since the scale of the system is small, it does not consume a lot of epochs to converge to the Pareto optimal. The superior performance of CQL

can be attributed to its ability to combine partial observations from multiple agents. Recall that unlike CQL where two partial observations are combined by a joint action learner and derive jointly observability, IQL agents do not communicate but act only based on their local observation. Therefore, they require more epochs to learn their policy through uncertainty.

5 Conclusions

As four conducted experiments demonstrated, CQL is guaranteed to converge to the solution of a smaller system with fewer epochs. However, the necessity of communication between the joint action learner and each agent and the exponentially growing action space significantly limits its application capability. IQL, on the other hand, does not have those limitations; however the environment dynamics caused by other agents interrupt the agent's convergence through uncertainty. These findings underscore the need for further improvements in IQL, particularly in addressing non-stationarity and partial observability. As Section 4 analyzed the performance difference between two based on the presence of communication between agents, this paper suggests allowing agent communication as a possible improvement of IQL. In such situations, the noise in the communication also has to be considered to simulate technical unreliability of communication, for instance, the noise of the sensor.

Overall, this study demonstrated the potential of reducing decentralized control systems to a single-agent RL framework as well as possible improvement for larger system application. Since single-agent RL has the advantage of simplicity, the future research should focus on comparing the performance of single-agent RL against MARL algorithms to evaluate the trade-offs between performance vs. simplicity.

Reference

[1] Oliehoek, Frans A. "Decentralized POMDPs," in *Reinforcement Learning State of Art*, edited by Marco. Wiering and Martin v. Otterlo. Springer Berlin, Heidelberg, 2012. pp. 471-503

[2] Brunton, Steve. "Model Based Reinforcement Learning: Policy Iteration, Value Iteration, and Dynamic Programming." YouTube, 2022.
https://www.youtube.com/watch?v=sJIFUTITfBc%5C&t=1365s.

[3] S. V. Albrecht, Filippos Christianos, and Lukas Schäfer, Multi-Agent Reinforcement Learning. MIT Press, pp. 33, 2024.

[4] Daniel Soper, "Foundations of Q-Learning," April 22, 2020. educational video,
https://www.youtube.com/watch?v=__t2XRxXGxI

[5] Ushio, Toshimitsu and Yamasaki, Tatsushi. "Decentralized Supervisory Control in Discrete Event Systems Based on Reinforcement Learning." 3049. In Transactions of the Institute of Systems Control and Information Engineers, ISCIE, November 2005.